

EXPERIMENT -7

Implementation of Iterative and Decision-Making Constructs Using Modular Programming in C

1. Activity Statement

Prepare an algorithm for each operation and a test-case table covering valid, invalid, and boundary inputs. Then develop, compile, and execute a menu-driven modular C project that accepts a positive integer and performs the specified mathematical and digit-based analyses. Test the program using the prepared test cases.

2. Algorithms of Operations

2.1 Count Digits

1. Start.
2. Initialize count = 0 to prevent RAM garbage values from corrupting the iteration.
3. Loop while number > 0.
4. Divide the number by 10 and increment count by 1.
5. Stop the loop and return count.
6. Stop.

2.2 Sum of Digits

1. Start.
2. Initialize sum = 0 explicitly.
3. Loop while number > 0.
4. Extract the last digit using number % 10 and add it to sum.
5. Remove the last digit using integer division (number / 10).
6. Stop the loop and return sum.
7. Stop.

2.3 Product of Digits

1. Start.
2. If `number == 0`, return 0.
3. Initialize `product = 1`.
4. Loop while `number > 0`.
5. Extract the last digit (`number % 10`) and multiply product by it.
6. Remove the last digit using division (`number / 10`).
7. Stop the loop and return product.
8. Stop.

2.4 Reverse Number

1. Start.
2. Initialize `reverse = 0`.
3. Loop while `number > 0`.
4. Update the variable using `reverse = reverse * 10 + (number % 10)`.
5. Remove the last digit using division (`number / 10`).
6. Stop the loop and return reverse.
7. Stop.

2.5 Largest Digit

1. Start.
2. Initialize largest with the value of the last digit (`number % 10`).
3. Divide number by 10.
4. Loop while `number > 0`.
5. Extract the current last digit. If it is greater than largest, update largest.
6. Divide number by 10.
7. Stop the loop and return largest.
8. Stop.

2.6 Smallest Digit

1. Start.
2. Initialize smallest with the value of the last digit ($\text{number} \% 10$).
3. Divide number by 10.
4. Loop while $\text{number} > 0$.
5. Extract the current last digit. If it is less than smallest, update smallest.
6. Divide number by 10.
7. Stop the loop and return smallest.
8. Stop.

2.7 Palindrome Test

1. Start.
2. Call `reverse_number(number)` and store the result.
3. Compare the reversed result with the original number.
4. If they match, return TRUE (1); otherwise, return FALSE (0).
5. Stop.

2.8 Armstrong Test

1. Start.
2. Initialize $\text{sum} = 0$.
3. Call `count_digits(number)` to find the total digits (exponent).
4. Copy the original number into a temporary variable.
5. Loop while the temporary variable is greater than 0.
6. Extract the last digit, calculate `integer_power(digit, exponent)`, and add it to sum.
7. Remove the last digit from the temporary variable.
8. If $\text{sum} == \text{number}$, return TRUE (1); otherwise, return FALSE (0).
9. Stop.

2.9 Strong Number Test

1. Start.
2. Initialize sum = 0.
3. Copy the original number into a temporary variable.
4. Loop while the temporary variable is greater than 0.
5. Extract the last digit, calculate factorial(digit), and add it to sum.
6. Remove the last digit from the temporary variable.
7. If sum == number, return TRUE (1); otherwise, return FALSE (0).
8. Stop.

2.10 Perfect Number Test

1. Start.
2. Initialize sum = 0.
3. Loop a variable i from 1 up to number / 2.
4. If number % i == 0, add i to sum.
5. If sum == number, return TRUE (1); otherwise, return FALSE (0).
6. Stop.

2.11 Prime Number Test

1. Start.
2. If number < 2, return FALSE (0).
3. Loop a divisor variable from 2 while divisor <= (number / divisor).
4. If number % divisor == 0, return FALSE (0).
5. If the loop finishes without finding any strict divisibility, return TRUE (1).
6. Stop.

2.12 Factorial

1. Start.
2. Initialize result = 1.
3. Loop a variable i from 2 up to the provided number.
4. Multiply result by i during each iteration.
5. Return result.
6. Stop.

2.13 Integer Power

1. Start.
2. Initialize result = 1.
3. Loop exactly exponent times.
4. Multiply result by base during each iteration.
5. Return result.
6. Stop.

2.14 Greatest Common Divisor (GCD)

1. Start.
2. Loop while second is not zero.
3. Store second temporarily, assign first % second to second, and assign the temporary value to first.
4. Return first as the GCD.
5. Stop.

2.15 Least Common Multiple (LCM)

1. Start.
2. Calculate the LCM by dividing first by `find_gcd(first, second)` and then multiplying by second to structurally prevent integer overflow.
3. Return the final calculation.
4. Stop.

3. Test Case table

Test case type	Input Number	Menu option	Additional Input	Expected Output
Invalid	-5	N/A	N/A	Input must be positivbe
Valid (Armstrong)	153	6	N/A	153 is a Armstrong number
Valid (Strong)	145	7	N/A	145 is a Strong number.
Valid (Perfect)	28	8	N/A	28 is a Perfect number.
Valid (Prime)	29	9	N/A	29 is a Prime number.
Valid (Palindrome)	1221	5	N/A	1221 is a Palindrome.
Boundary (Largest/Smallest)	904	10&11	N/A	Largest: 9, Smallest: 0
Valid (GCD & LCM)	12	12	18	GCD: 6, LCM: 36.

4.The C codes

4.1 number.h

```
#ifndef NUMBER_H
#define NUMBER_H

#define TRUE 1
#define FALSE 0

int count_digits(int number);
int sum_of_digits(int number);
int product_of_digits(int number);
int reverse_number(int number);
int largest_digit(int number);
int smallest_digit(int number);

int is_palindrome(int number);
int is_armstrong(int number);
int is_strong(int number);
int is_perfect(int number);
int is_prime(int number);

int factorial(int number);
int integer_power(int base, int exponent);
int find_gcd(int first, int second);
int find_lcm(int first, int second);

#endif
```

4.2 digit_operations.c

```
#include "number.h"
```

```
int count_digits(int number) {  
    int count = 0;  
    for (; number > 0; number /= 10) {  
        count++;  
    }  
    return count;  
}
```

```
int sum_of_digits(int number) {  
    int sum = 0;  
    for (; number > 0; number /= 10) {  
        sum += number % 10;  
    }  
    return sum;  
}
```

```
int product_of_digits(int number) {  
    if (number == 0) return 0;  
    int product = 1;  
    for (; number > 0; number /= 10) {  
        product *= (number % 10);  
    }  
    return product;  
}
```

```
int reverse_number(int number)
{
    int reverse = 0;
    for (; number > 0; number /= 10)
    {
        reverse = reverse * 10 + (number % 10);
    }
    return reverse;
}
```

```
int largest_digit(int number)
{
    int largest = number % 10;
    for (number /= 10; number > 0; number /= 10)
    {
        if (number % 10 > largest)
        {
            largest = number % 10;
        }
    }
    return largest;
}
```

```
int smallest_digit(int number)
{
    int smallest = number % 10;
    for (number /= 10; number > 0; number /= 10)
    {
        if (number % 10 < smallest)
        {
            smallest = number % 10;
        }
    }
    return smallest;
}
```

4.3 number_properties.c

```
#include "number.h"

int is_palindrome(int number) {
    return (number == reverse_number(number)) ? TRUE : FALSE;
}

int is_armstrong(int number) {
    int sum = 0;
    int digits = count_digits(number);
    int temp = number;
    for (; temp > 0; temp /= 10) {
        int digit = temp % 10;
        sum += integer_power(digit, digits);
    }
    return (sum == number) ? TRUE : FALSE;
}

int is_strong(int number) {
    int sum = 0;
    int temp = number;
    for (; temp > 0; temp /= 10) {
        int digit = temp % 10;
        sum += factorial(digit);
    }
    return (sum == number) ? TRUE : FALSE;
}

int is_perfect(int number) {
    int sum = 0;
    for (int i = 1; i <= number / 2; i++) {
        if (number % i == 0) {
            sum += i;
        }
    }
    return (sum == number) ? TRUE : FALSE;
}

int is_prime(int number) {
    if (number < 2) return FALSE;
    for (int divisor = 2; divisor <= number / divisor; divisor++) {
        if (number % divisor == 0) {
            return FALSE;
        }
    }
    return TRUE;
}
```

4.4 number_utils.c

```
#include "number.h"
```

```
int factorial(int number) {  
    int result = 1;  
    for (int i = 2; i <= number; i++) {  
        result *= i;  
    }  
    return result;  
}
```

```
int integer_power(int base, int exponent) {  
    int result = 1;  
    for (int i = 0; i < exponent; i++) {  
        result *= base;  
    }  
    return result;  
}
```

```
int find_gcd(int first, int second) {  
    while (second != 0) {  
        int temp = second;  
        second = first % second;  
        first = temp;  
    }  
    return first;  
}
```

```
int find_lcm(int first, int second) {  
    return (first / find_gcd(first, second)) * second;  
}
```

4.5 THE MAIN C

```
#include <stdio.h>
#include "number.h"

int main() {
    int number, choice;

    do {
        printf("Enter a positive integer: ");
        scanf("%d", &number);
        if (number <= 0) {
            printf("Error: Input must be positive. Please try again.\n");
        }
    } while (number <= 0);

    do {
        printf("\n--- Mathematical & Digit Analysis Menu ---\n");
        printf("1. Count the digits\n");
        printf("2. Find the sum of the digits\n");
        printf("3. Find the product of the digits\n");
        printf("4. Reverse the number\n");
        printf("5. Check whether it is a palindrome\n");
        printf("6. Check whether it is an Armstrong number\n");
        printf("7. Check whether it is a Strong number\n");
        printf("8. Check whether it is a Perfect number\n");
        printf("9. Check whether it is Prime\n");
        printf("10. Find the largest digit\n");
        printf("11. Find the smallest digit\n");
        printf("12. Find GCD and LCM\n");
        printf("13. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
```

```
switch (choice) {
case 1:
printf("Number of digits: %d\n", count_digits(number));
break;
case 2:
printf("Sum of digits: %d\n", sum_of_digits(number));
break;
case 3:
printf("Product of digits: %d\n",
product_of_digits(number));
break;
case 4:
printf("Reversed number: %d\n",
reverse_number(number));
break;
case 5:
if (is_palindrome(number) == TRUE) printf("%d is a
Palindrome.\n", number);
else printf("%d is not a Palindrome.\n", number);
break;
case 6:
if (is_armstrong(number) == TRUE) printf("%d is an
Armstrong number.\n", number);
else printf("%d is not an Armstrong number.\n",
number);
break;
case 7:
if (is_strong(number) == TRUE) printf("%d is a Strong
number.\n", number);
else printf("%d is not a Strong number.\n", number);
break;
case 8:
if (is_perfect(number) == TRUE) printf("%d is a Perfect
number.\n", number);
else printf("%d is not a Perfect number.\n", number);
break;
```

```

case 9:
if (is_prime(number) == TRUE) printf("%d is a Prime
number.\n", number);
else printf("%d is not a Prime number.\n", number);
break;
case 10:
printf("Largest digit: %d\n", largest_digit(number));
break;
case 11:
printf("Smallest digit: %d\n", smallest_digit(number));
break;
case 12: {
int second;
printf("Enter another positive integer: ");
scanf("%d", &second);
if (second > 0) {
printf("GCD: %d\n", find_gcd(number, second));
printf("LCM: %d\n", find_lcm(number, second));
} else {
printf("Error: Second number must be positive.\n");
}
break;
}
case 13:
printf("Exiting program.\n");
break;
default:
printf("Invalid choice! Please select an option from 1 to
13.\n");
}
} while (choice != 13);

return 0;
}

```

5. Output

case : invalid

```
(kali㉿kali)-[~/codes/exp_7]
└─$ ls
digit_operations.c  main.c  number.h  number_properties.c  number_utils.c

(kali㉿kali)-[~/codes/exp_7]
└─$ gcc main.c digit_operations.c number_properties.c number_utils.c -o experiment_7

(kali㉿kali)-[~/codes/exp_7]
└─$ ./experiment_7
Enter a positive integer: -5
Error: Input must be positive. Please try again.
```

case : valid armstrong

```
Enter a positive integer: 153

— Mathematical & Digit Analysis Menu —
1. Count the digits
2. Find the sum of the digits
3. Find the product of the digits
4. Reverse the number
5. Check whether it is a palindrome
6. Check whether it is an Armstrong number
7. Check whether it is a Strong number
8. Check whether it is a Perfect number
9. Check whether it is Prime
10. Find the largest digit
11. Find the smallest digit
12. Find GCD and LCM
13. Exit
Enter your choice: 6
153 is an Armstrong number.

— Mathematical & Digit Analysis Menu —
1. Count the digits
2. Find the sum of the digits
3. Find the product of the digits
4. Reverse the number
5. Check whether it is a palindrome
6. Check whether it is an Armstrong number
7. Check whether it is a Strong number
8. Check whether it is a Perfect number
9. Check whether it is Prime
10. Find the largest digit
11. Find the smallest digit
12. Find GCD and LCM
13. Exit
Enter your choice: 13
Exiting program.
```

case : valid strong

```
Enter a positive integer: 145
```

```
— Mathematical & Digit Analysis Menu —
```

1. Count the digits
2. Find the sum of the digits
3. Find the product of the digits
4. Reverse the number
5. Check whether it is a palindrome
6. Check whether it is an Armstrong number
7. Check whether it is a Strong number
8. Check whether it is a Perfect number
9. Check whether it is Prime
10. Find the largest digit
11. Find the smallest digit
12. Find GCD and LCM
13. Exit

```
Enter your choice: 7
```

```
145 is a Strong number.
```

```
— Mathematical & Digit Analysis Menu —
```

1. Count the digits
2. Find the sum of the digits
3. Find the product of the digits
4. Reverse the number
5. Check whether it is a palindrome
6. Check whether it is an Armstrong number
7. Check whether it is a Strong number
8. Check whether it is a Perfect number
9. Check whether it is Prime
10. Find the largest digit
11. Find the smallest digit
12. Find GCD and LCM
13. Exit

```
Enter your choice: 13
```

```
Exiting program.
```

case : valid perfect

```
└─$ ./experiment_7
Enter a positive integer: 28

— Mathematical & Digit Analysis Menu —
1. Count the digits
2. Find the sum of the digits
3. Find the product of the digits
4. Reverse the number
5. Check whether it is a palindrome
6. Check whether it is an Armstrong number
7. Check whether it is a Strong number
8. Check whether it is a Perfect number
9. Check whether it is Prime
10. Find the largest digit
11. Find the smallest digit
12. Find GCD and LCM
13. Exit
Enter your choice: 8
28 is a Perfect number.

— Mathematical & Digit Analysis Menu —
1. Count the digits
2. Find the sum of the digits
3. Find the product of the digits
4. Reverse the number
5. Check whether it is a palindrome
6. Check whether it is an Armstrong number
7. Check whether it is a Strong number
8. Check whether it is a Perfect number
9. Check whether it is Prime
10. Find the largest digit
11. Find the smallest digit
12. Find GCD and LCM
13. Exit
Enter your choice: 13
Exiting program.
```

case : valid prime

```
└─$ ./experiment_7
Enter a positive integer: 29

— Mathematical & Digit Analysis Menu —
1. Count the digits
2. Find the sum of the digits
3. Find the product of the digits
4. Reverse the number
5. Check whether it is a palindrome
6. Check whether it is an Armstrong number
7. Check whether it is a Strong number
8. Check whether it is a Perfect number
9. Check whether it is Prime
10. Find the largest digit
11. Find the smallest digit
12. Find GCD and LCM
13. Exit
Enter your choice: 9
29 is a Prime number.

— Mathematical & Digit Analysis Menu —
1. Count the digits
2. Find the sum of the digits
3. Find the product of the digits
4. Reverse the number
5. Check whether it is a palindrome
6. Check whether it is an Armstrong number
7. Check whether it is a Strong number
8. Check whether it is a Perfect number
9. Check whether it is Prime
10. Find the largest digit
11. Find the smallest digit
12. Find GCD and LCM
13. Exit
Enter your choice: 13
Exiting program.
```

case : valid Palindrome

```
└─$ ./experiment_7
Enter a positive integer: 1221

— Mathematical & Digit Analysis Menu —
1. Count the digits
2. Find the sum of the digits
3. Find the product of the digits
4. Reverse the number
5. Check whether it is a palindrome
6. Check whether it is an Armstrong number
7. Check whether it is a Strong number
8. Check whether it is a Perfect number
9. Check whether it is Prime
10. Find the largest digit
11. Find the smallest digit
12. Find GCD and LCM
13. Exit
Enter your choice: 5
1221 is a Palindrome.

— Mathematical & Digit Analysis Menu —
1. Count the digits
2. Find the sum of the digits
3. Find the product of the digits
4. Reverse the number
5. Check whether it is a palindrome
6. Check whether it is an Armstrong number
7. Check whether it is a Strong number
8. Check whether it is a Perfect number
9. Check whether it is Prime
10. Find the largest digit
11. Find the smallest digit
12. Find GCD and LCM
13. Exit
Enter your choice: 13
Exiting program.
```

case : valid Boundary (Largest/Smallest)

```
(kali@kali)-[~/codes/exp_7]
$ ./experiment_7
Enter a positive integer: 904

— Mathematical & Digit Analysis Menu —
1. Count the digits
2. Find the sum of the digits
3. Find the product of the digits
4. Reverse the number
5. Check whether it is a palindrome
6. Check whether it is an Armstrong number
7. Check whether it is a Strong number
8. Check whether it is a Perfect number
9. Check whether it is Prime
10. Find the largest digit
11. Find the smallest digit
12. Find GCD and LCM
13. Exit
Enter your choice: 10
Largest digit: 9

— Mathematical & Digit Analysis Menu —
1. Count the digits
2. Find the sum of the digits
3. Find the product of the digits
4. Reverse the number
5. Check whether it is a palindrome
6. Check whether it is an Armstrong number
7. Check whether it is a Strong number
8. Check whether it is a Perfect number
9. Check whether it is Prime
10. Find the largest digit
11. Find the smallest digit
12. Find GCD and LCM
13. Exit
Enter your choice: 11

Enter your choice: 11
Smallest digit: 0

— Mathematical & Digit Analysis Menu —
1. Count the digits
2. Find the sum of the digits
3. Find the product of the digits
4. Reverse the number
5. Check whether it is a palindrome
6. Check whether it is an Armstrong number
7. Check whether it is a Strong number
8. Check whether it is a Perfect number
9. Check whether it is Prime
10. Find the largest digit
11. Find the smallest digit
12. Find GCD and LCM
13. Exit
Enter your choice: 13
Exiting program.
```

Case : Valid (GCD & LCM)

```
(kali@kali)-[~/codes/exp_7]
└─$ ./experiment_7
Enter a positive integer: 12

— Mathematical & Digit Analysis Menu —
1. Count the digits
2. Find the sum of the digits
3. Find the product of the digits
4. Reverse the number
5. Check whether it is a palindrome
6. Check whether it is an Armstrong number
7. Check whether it is a Strong number
8. Check whether it is a Perfect number
9. Check whether it is Prime
10. Find the largest digit
11. Find the smallest digit
12. Find GCD and LCM
13. Exit
Enter your choice: 12
Enter another positive integer: 18
GCD: 6
LCM: 36

— Mathematical & Digit Analysis Menu —
1. Count the digits
2. Find the sum of the digits
3. Find the product of the digits
4. Reverse the number
5. Check whether it is a palindrome
6. Check whether it is an Armstrong number
7. Check whether it is a Strong number
8. Check whether it is a Perfect number
9. Check whether it is Prime
10. Find the largest digit
11. Find the smallest digit
12. Find GCD and LCM
13. Exit
Enter your choice: 13
Exiting program.
```